

数据结构记忆背诵

图

以下是关于数据结构中图的基本概念和知识：

图的定义

图是由顶点（Vertex）集合和边（Edge）集合组成的数据结构，通常表示为 $G=(V,E)$ ，其中 G 表示图， V 是顶点的集合， E 是边的集合。

图的存储结构

- 邻接矩阵**：用一个二维数组来表示图中顶点之间的邻接关系。对于无向图，邻接矩阵是对称的；对于有向图，邻接矩阵不一定对称。如果图中边带有权重，通常用一个特殊的值（如0或 ∞ ）表示不存在边，其他值表示边的权重。
- 邻接表**：对图中的每个顶点建立一个链表，链表中的节点表示与该顶点相邻的顶点以及边的信息（如权重等）。对于有向图，每个顶点的链表存储的是该顶点的出边信息；对于无向图，每个顶点的链表存储的是与该顶点相连的边的信息。

图的遍历

- 深度优先搜索（DFS）**：从图中的某个顶点开始，沿着一条路径尽可能深地访问顶点，直到无法继续或访问完所有顶点，然后回溯到前一个顶点，继续探索其他路径，直到访问完所有顶点。
- 广度优先搜索（BFS）**：从图中的某个顶点开始，先访问该顶点的所有邻接顶点，然后再依次访问这些邻接顶点的邻接顶点，直到访问完所有顶点。

图的应用

- 最短路径问题**：如Dijkstra算法、Floyd算法等，用于在带权图中找到两个顶点之间的最短路径。
- 最小生成树问题**：如Prim算法、Kruskal算法等，用于在连通带权无向图中找到一棵最小生成树，即连接所有顶点且边权之和最小的树。
- 拓扑排序**：用于对有向无环图（DAG）中的顶点进行排序，使得对于图中的任意一条有向边 $((u, v))$ ，顶点 u 在排序中都出现在顶点 v 之前。常用于任务调度、课程安排等场景。

邻接矩阵

```
#define MAXNUMBER 20
typedef VRTYPE int
typedef InfoType *info

typedef struct ArcCell{
    VRTYPE adj;
    InfoType *info
}ArcCell,AdjMaxtric[MAXNUMBER][MAXNUMBER];

typedef struct {
    char vexs[MAXNUMBER];
    AdjMaxtric arcs;
    int vexnum,arcnum;
}MGraph
```

邻接矩阵

```
#define MAXNUM 20
typedef int InfoType;
typedef char VertexType;

typedef struct ArcNode{
    int adjvex;
    struct ArcNode *next;
    InfoType *info;
}ArcNode;

typedef struct VNode{
    int data;
    ArcNode *firstarc;
}VNode,AdjList[MAXNUM];

typedef struct {
    AdjList vertices;
    int vexnum,arcnum;
}ALGraph;
```

十字链表

```

#define MAX_VERTEX_NUM 20

// 边节点结构
typedef struct ArcNode {
    int tailvex, headvex;
    struct ArcNode *hlink, *tlink;
} ArcNode;

// 顶点节点结构
typedef struct VexNode {
    int data;
    ArcNode *firstin, *firstout;
} VexNode, AdjList[MAX_VERTEX_NUM];

// 十字链表结构
typedef struct {
    AdjList vertices;
    int vexnum, arcnum;
} OLGraph;

```

邻接多重表

```

#define MAX_VERTEX_NUM 20

// 边节点结构
typedef struct EBox {
    int ivex, jvex;
    struct EBox *ilink, *jlink;
} EBox;

// 顶点节点结构
typedef struct VexBox {
    char data;
    EBox *firstedge;
} VexBox, AdjList[MAX_VERTEX_NUM];

// 邻接多重表结构
typedef struct {
    AdjList adjmulist;
    int vexnum, edgenum;
} AMLGraph;

```

树

树的存储结构

顺序存储结构

```
#define MAX_TREE_SIZE 100

typedef struct {
    int data[MAX_TREE_SIZE];
    int n;
} SeqTree;
```

链式存储结构

```
typedef int TElemType;

typedef struct BiTNode {
    TElemType data;
    struct BiTNode *lchild, *rchild;
} BiTNode, *BiTree;
```

线索二叉树

```
typedef enum PointerTag { Link, Thread};
typedef struct BiThrNode {
    TElemType data;
    PointerTag LTag, RTag;
    struct BiThrNode *lchild, *rchild;
} BiThrNode, *BiThrTree;

typedef enum Pointtype{link,thread}
typedef struct BiThrNode{
    int data;
    Pointtype Ltag,Rtag;
    struct BiThrNode *lchild,*rchild;
}BiThrNode;
```

树的双亲表示法

```

#define MAX_TREE_SIZE 100
typedef struct PTNode {    //节点结构
    TElemType data;
    int parent;    //双亲位置域
}PTNode;
typedef struct {    //树结构
    PTNode nodes[MAX_TREE_SIZE];
    int r, n;    //根的位置和节点数
}PTree;

```

树的孩子表示法

```

// 定义孩子节点结构体
typedef struct CNode {    // 孩子节点
    int child;    // 孩子节点的编号
    struct CNode *next;    // 指向下一个孩子节点的指针
} *ChildPtr;

// 定义树中每个节点的结构体
typedef struct {
    TElemType data;    // 节点的数据，TElemType 需根据实际情况定义
    ChildPtr firstchild;    // 孩子链表头指针，指向该节点的第一个孩子节点
} CBox;

// 定义整棵树的结构体
typedef struct {
    CBox nodes[MAX_TREE_SIZE];    // 存储树中所有节点的数组
    int n, r;    // n 表示树的节点数量，r 表示根节点的编号
} CTree;

```

树的孩子 - 兄弟表示法

```

// 定义树节点结构体
typedef struct CNode
{
    ElemType data;    // 节点存储的数据，ElemType 需根据实际情况定义
    struct CNode *firstchild;    // 指向该节点的第一个孩子节点
    struct CNode *nextsibling;    // 指向该节点的下一个兄弟节点
} CNode, *CSTree;

```

非递归遍历

// 非递归前序遍历

```
void preorderTraversal(TreeNode* root) {  
    if (root == NULL) return;  
    Stack* stack = createStack(100);  
    push(stack, root);  
  
    while (!isEmpty(stack)) {  
        TreeNode* node = pop(stack);  
        printf("%d ", node->data);  
  
        if (node->right)  
            push(stack, node->right);  
        if (node->left)  
            push(stack, node->left);  
    }  
    free(stack->array);  
    free(stack);  
}
```

进栈 Stack[++top]=e;

出栈 e=Stack[top--];

// 非递归中序遍历

```
void inorderTraversal(TreeNode* root) {  
    Stack* stack = createStack(100);  
    TreeNode* current = root;  
  
    while (current != NULL || !isEmpty(stack)) {  
        while (current != NULL) {  
            push(stack, current);  
            current = current->left;  
        }  
  
        current = pop(stack);  
        printf("%d ", current->data);  
        current = current->right;  
    }  
    free(stack->array);  
    free(stack);  
}
```

// 非递归后序遍历

```
void postorderTraversal(TreeNode* root) {  
    if (root == NULL) return;  
    Stack* stack1 = createStack(100);  
    Stack* stack2 = createStack(100);  
    push(stack1, root);  
  
    while (!isEmpty(stack1)) {  
        TreeNode* node = pop(stack1);
```

```

        push(stack2, node);

        if (node->left)
            push(stack1, node->left);
        if (node->right)
            push(stack1, node->right);
    }

    while (!isEmpty(stack2)) {
        TreeNode* node = pop(stack2);
        printf("%d ", node->data);
    }
    free(stack1->array);
    free(stack1);
    free(stack2->array);
    free(stack2);
}

#include <stdio.h>
#include <stdlib.h>

// 定义树节点结构
typedef struct TreeNode {
    int data;
    struct TreeNode* left;
    struct TreeNode* right;
} TreeNode;

// 定义栈结构
typedef struct Stack {
    TreeNode** array;
    int top;
    int capacity;
} Stack;

// 创建栈
Stack* createStack(int capacity) {
    Stack* stack = (Stack*)malloc(sizeof(Stack));
    stack->array = (TreeNode**)malloc(capacity * sizeof(TreeNode*));
    stack->top = -1;
    stack->capacity = capacity;
    return stack;
}

// 判断栈是否为空
int isEmpty(Stack* stack) {
    return stack->top == -1;
}

```

```

// 进栈操作
void push(Stack* stack, TreeNode* node) {
    stack->array[++(stack->top)] = node;
}

// 出栈操作
TreeNode* pop(Stack* stack) {
    return stack->array[(stack->top)--];
}

// 非递归前序遍历
void preorderTraversal(TreeNode* root) {
    if (root == NULL) return;
    // 填空1: 创建一个容量为100的栈
    Stack* stack = _____; createStack[100]
    // 填空2: 将根节点压入栈
    _____; push(stack, root)

    while (!isEmpty(stack)) {
        // 填空3: 从栈中弹出一个节点
        TreeNode* node = _____; pop(stack)
        printf("%d ", node->data);

        if (_____ ) node->right
            push(stack, _____); node->right
        if (_____ ) node->left
            push(stack, _____); node->left
    }
    // 填空4: 释放栈的内存
    _____; free
}

// 非递归中序遍历
void inorderTraversal(TreeNode* root) {
    // 填空5: 创建一个容量为100的栈
    Stack* stack = _____;
    TreeNode* current = root;

    while (current != NULL || _____ isEmpty(stack) ) {
        while (current != NULL) {
            // 填空6: 将当前节点压入栈
            _____; push(stack, current)
            current = current->left;
        }

        // 填空7: 从栈中弹出一个节点
        current = _____; pop(stack)
        printf("%d ", current->data);
        current = current->right;
    }
    // 填空8: 释放栈的内存

```



```

        _____;free
    }

// 非递归后序遍历
void postorderTraversal(TreeNode* root) {
    if (root == NULL) return;
    // 填空9: 创建第一个容量为100的栈
    Stack* stack1 = _____;createStack(100)
    // 填空10: 创建第二个容量为100的栈
    Stack* stack2 = _____;createStack(100)
    // 填空11: 将根节点压入第一个栈
    _____;push(stack1)

    while (!isEmpty(stack1)) {
        // 填空12: 从第一个栈中弹出一个节点并压入第二个栈
        TreeNode* node = _____;pop(stack1)
        push(stack2, node);

        if (node->right)
            push(stack1, node->right);
        if (node->left)
            push(stack1, node->left);
    }

    while (!isEmpty(stack2)) {
        // 填空13: 从第二个栈中弹出一个节点
        TreeNode* node = _____;pop(stack2)
        printf("%d ", node->data);
    }
    // 填空14: 释放两个栈的内存
    _____;
}

int main() {
    // 这里可以构建树并调用遍历函数进行测试
    return 0;
}

void PostOrder(BiTree){ // 全篇♥
    InitStack(s);
    BiTree *p=T,*r=NULL; // r标记最近访问过的结点
    while(p!=NULL || !IsEmpty(s)){
        if(p!=NULL){
            push(s,p); // 一直向左走, 左孩子入栈
            p=p->lchild;
        }
        else{
            GetTop(s,p);
            // 获取s的栈顶元素赋值给p
            // GetTop(s,p)意思就是判断栈顶元素的情况
            // ♥case one♥

```

```

        if(p->rchild && p->rchild!=r){
            // 若右孩子存在且未被访问
            p=p->rchild; // 就让右孩子
            push(s,p) // 入栈
            p=p->lchild; // 让右孩子向左
            //上面三句意思就是让右孩子的左孩子一直入栈，一直向左走
        }
        // ♥case two♥
        else{
            pop(s,p); // 右孩子为空或未被访问过，就出栈
            visit(p->data);
            r=p; // r标记最近访问结点
            p=NULL; // r置空
            // 置空原因：因为这个结点已经出栈了
            //继续指向就没必要了，置空后r不标记任何结点
        }
    }
}

void LevelorderTraverse( BTreeNode *T) {
    BTreeNode *Queue[MAX_NODE], *p = T;
    int front = 0, rear = 0;

    if (p != NULL) {
        Queue[++rear] = p; /* 根结点入队 */
        while (front < rear) {
            p = Queue[++front];
            printf("%d ", p->data); // 访问节点数据
            if (p->Lchild != NULL)
                Queue[++rear] = p->Lchild; /* 左结点入队 */
            if (p->Rchild != NULL)
                Queue[++rear] = p->Rchild; /* 右结点入队 */
        }
    }
}

if (p!=NULL){
    Queue[++rear]=p;
    while(front<rear){
        p=Queue[++front]
        printf("%d",p->data);
        if (p->Lchild!=NULL){
            Queue[++rear] = p->Lchild; /* 左结点入队 */
        }
    }
}
}

#define MAX_NODE 50
int search_leaves(BTreeNode *T){

```

```

    BTreeNode *Stack[MAXLENTH], *p=T;
    int top=0, num=0;
    if (T!=NULL){
        Stack[++top]=p;
        while(top>0){
            p=Stack[top--];
            if (p->Lchild&& p->Rchild){
                num++;
                if (p->Rchild!=NULL )
                    Stack[++top]=p->Rchild;
                // 再压入左子节点
                if (p->Lchild!=NULL )
                    Stack[++top]=p->Lchild;
            }
        }
        return num;
    }

int search_depth(BTreeNode *T) {
    BTreeNode *Queue[MAX_NODE], *p = T;
    int front = 0, rear = 0, depth = 0, level;
    /* level总是指向访问层的最后一个结点在队列的位置 */
    if (T != NULL) {
        Queue[++rear] = p;    /* 根结点入队 */
        level = rear;    /* 根是第1层的最后一个节点 */
        while (front < rear) {
            p = Queue[++front];
            if (p->Lchild != NULL)
                Queue[++rear] = p->Lchild;    /* 左结点入队 */
            if (p->Rchild != NULL)
                Queue[++rear] = p->Rchild;    /* 右结点入队 */
            if (front == level)
                /* 正访问的是当前层的最后一个结点 */
                {
                    depth++;
                    level = rear;
                }
        }
    }
    return depth;
}

// 求二叉树的结点总数
int countNodes(TreeNode* root) {
    if (root == NULL) {
        return 0;
    }
    return 1 + countNodes(root->left) + countNodes(root->right);
}

```

```

// 求二叉树的叶子结点总数
int countLeaves(TreeNode* root) {
    if (root == NULL) {
        return 0;
    }
    if (root->left == NULL && root->right == NULL) {
        return 1;
    }
    return countLeaves(root->left) + countLeaves(root->right);
}

// 前序遍历：根节点 -> 左子树 -> 右子树
void preOrderTraversal(TreeNode* root) {
    if (root == NULL) {
        return;
    }
    printf("%d ", root->data);
    preOrderTraversal(root->left);
    preOrderTraversal(root->right);
}

// 中序遍历：左子树 -> 根节点 -> 右子树
void inOrderTraversal(TreeNode* root) {
    if (root == NULL) {
        return;
    }
    inOrderTraversal(root->left);
    printf("%d ", root->data);
    inOrderTraversal(root->right);
}

// 后序遍历：左子树 -> 右子树 -> 根节点
void postOrderTraversal(TreeNode* root) {
    if (root == NULL) {
        return;
    }
    postOrderTraversal(root->left);
    postOrderTraversal(root->right);
    printf("%d ", root->data);
}

// 交换二叉树中所有结点的左、右子树
TreeNode* swapSubtrees(TreeNode* root) {
    if (root == NULL) {
        return NULL;
    }
    // 交换当前节点的左右子树
    TreeNode* temp = root->left;
    root->left = root->right;
    root->right = temp;
}

```

```

    // 递归交换左子树
    swapSubtrees(root->left);
    // 递归交换右子树
    swapSubtrees(root->right);

    return root;
}

// 求树的深度
int treeDepth(TreeNode* root) {
    if (root == NULL) {
        return 0;
    }
    int leftDepth = treeDepth(root->left);
    int rightDepth = treeDepth(root->right);
    return (leftDepth > rightDepth ? leftDepth : rightDepth) + 1;
}

```

线性表

顺序

```

//顺序
typedef struct {
    int data[MAX_SIZE];
    int length;
} SeqList;

// 定义链表节点结构体
typedef struct Node {
    int data;
    struct Node *next;
} Node;

// 定义双向链表节点结构体
typedef struct Node {
    int data;
    struct Node *prev;
    struct Node *next;
} Node;

```

// 将所有在线性表 Lb 中但不在 La 中的数据元素插入到 La 中

```
void unionLists(List *La, List Lb) {  
    int La_len = ListLength(*La);  
    int Lb_len = ListLength(Lb);  
    int e;  
    for (int i = 1; i <= Lb_len; i++) {  
        if (GetElem(Lb, i, &e)) {  
            if (!LocateElem(*La, e)) {  
                ListInsert(La, ++La_len, e);  
            }  
        }  
    }  
}
```

Status Insert_SqList(SqList *L, int i, ElemType e)

```
{  
    int j;  
  
    // 检查插入位置的合法性  
    if (i < 1 || i > L->length + 1)  
        return ERROR;  
  
    // 检查顺序表是否已满  
    if (L->length >= MAX_SIZE)  
    {  
        printf("线性表溢出!\n");  
        return ERROR;  
    }  
  
    // 将i-1位置以后的所有结点后移  
    for (j = L->length - 1; j >= i - 1; --j)  
        L->Elem_array[j + 1] = L->Elem_array[j];  
  
    // 在i - 1位置插入结点  
    L->Elem_array[i - 1] = e;  
  
    // 顺序表长度加1  
    L->length++;  
  
    return OK;  
}
```

ElemType Delete_SqList(SqList *L, int i)

```
{  
    int k;  
    ElemType x;  
  
    // 检查顺序表是否为空  
    if (L->length == 0)  
    {
```

```

        printf("线性表L为空!\n");
        return ERROR;
    }
    // 检查删除位置的合法性
    else if (i < 1 || i > L->length)
    {
        printf("要删除的数据元素不存在!\n");
        return ERROR;
    }
    else
    {
        // 保存要删除结点的值
        x = L->Elem_array[i - 1];

        // 将i位置以后的所有结点前移
        for (k = i; k < L->length; k++)
            L->Elem_array[k - 1] = L->Elem_array[k];

        // 顺序表长度减1
        L->length--;

        // 返回被删除元素的值
        return (x);
    }
}

Status Locate_Delete_SqList(SqList *L, ElemType x)
{
    int i = 0, k, m = L->length;
    while (i < L->length)
    {
        if (L->Elem_array[i] != x)
            i++;
        else
        {
            for (k = i + 1; k < L->length; k++)
                L->Elem_array[k - 1] = L->Elem_array[k];
            L->length--;
            break;
        }
    }
    if (i >= m)
    {
        printf("要删除的数据元素不存在!\n");
        return ERROR;
    }
    return OK;
}

void Delete_Node_value(LNode *L)
/* 删除以L为头结点的单链表中所有值相同的结点 */

```

```

{
    LNode *p = L->next, *q, *ptr;
    // 检查链表中所有结点
    while (p != NULL)
    {
        q = p;
        ptr = p->next;
        // 检查结点p的所有后继结点ptr
        while (ptr != NULL)
        {
            if (ptr->data == p->data)
            {
                // 如果ptr结点的值和p结点的值相同，删除ptr结点
                q->next = ptr->next;
                free(ptr);
                ptr = q->next;
            }
            else
            {
                // 如果ptr结点的值和p结点的值不同，继续往后遍历
                q = ptr;
                ptr = ptr->next;
            }
        }
        // 处理完p结点的所有后继结点后，移动到下一个结点
        p = p->next;
    }
}

// 7. 求单链表的结点数目
int ListLength(ListNode *L) {
    int count = 0;
    ListNode *current = L;
    while (current) {
        count++;
        current = current->next;
    }
    return count;
}

// 8. 删除单链表中值重复的结点
ListNode* remove_duplicates(ListNode *L) {
    if (L == NULL) {
        return L;
    }
    ListNode *prev = L;
    ListNode *current = L->next;
    // 用于记录已经出现过的值
    int *visited = (int *)malloc(10000 * sizeof(int)); // 假设值的范围不太大，这里开辟一个足够大的数组
    for (int i = 0; i < 10000; i++) {

```



```

        visited[i] = 0;
    }
    visited[prev->val] = 1;
    while (current) {
        if (visited[current->val]) {
            prev->next = current->next;
            free(current);
            current = prev->next;
        } else {
            visited[current->val] = 1;
            prev = current;
            current = current->next;
        }
    }
    free(visited);
    return L;
}

```

// 9. 从一给定的向量（数组）A 删除值在 x 到 y($x \leq y$)之间的所有元素

```

void remove_range(int A[], int *size, int x, int y) {
    int i, j = 0;
    for (i = 0; i < *size; i++) {
        if (A[i] < x || A[i] > y) {
            A[j++] = A[i];
        }
    }
    *size = j;
}

```

栈和队列

```
// 定义链栈节点结构体
typedef struct StackNode {
    int data;
    struct StackNode *next;
} StackNode;

// 定义链栈结构体
typedef struct {
    StackNode *top;
} LinkedStack;

// 定义链队列节点结构体
typedef struct QueueNode {
    int data;
    struct QueueNode *next;
} QueueNode;

// 定义链队列结构体
typedef struct {
    QueueNode *front;
    QueueNode *rear;
} LinkedQueue;

// 定义顺序队列结构体
typedef struct {
    int data[MAX_SIZE];
    int front;
    int rear;
} SeqQueue;

// 定义顺序栈结构体
typedef struct {
    int data[MAX_SIZE];
    int top;
} SeqStack;
```

```

#include <stdio.h>
#include <stdlib.h>

// 栈的静态顺序存储表示
#define MAX_STACK_SIZE 100
typedef int ElemType;
typedef struct sqstack {
    ElemType stack_array[MAX_STACK_SIZE];
    int top;
} SqStack;

// 栈的初始化
SqStack Init_Stack(void) {
    SqStack S;
    S.top = 0;
    return (S);
}

// 压栈(元素进栈)
int push(SqStack *S, ElemType e) {
    if (S->top == MAX_STACK_SIZE)
        return 0;
    S->stack_array[S->top] = e;
    S->top++;
    return 1;
}

// 弹栈(元素出栈)
int pop(SqStack *S, ElemType *e) {
    if (S->top == 0)
        return 0;
    S->top--;
    *e = S->stack_array[S->top];
    return 1;
}

// 数制转换
void conversion(int n, int d) {
    SqStack S = Init_Stack();
    int k;
    while (n > 0) {
        k = n % d;
        push(&S, k);
        n = n / d;
    }
    while (S.top != 0) {
        pop(&S, &k);
        printf("%1d", k);
    }
    printf("\n");
}

```

```

}

// 括号的匹配
int Match_Brackets() {
    SqStack S = Init_Stack();
    char ch, x;
    while ((ch = getchar()) != '\n') {
        if ((ch == '(') || (ch == '['))
            push(&S, ch);
        else if (ch == ']') {
            if (!pop(&S, &x) || x != '[') {
                printf("'']' 括号不匹配\n");
                return 0;
            }
        } else if (ch == ')') {
            if (!pop(&S, &x) || x != '(') {
                printf("'')' 括号不匹配\n");
                return 0;
            }
        }
    }
    if (S.top != 0) {
        printf("括号数量不匹配! \n");
        return 0;
    }
    return 1;
}

// 行编辑程序
#define STACK_INIT_SIZE 100
#define STACKINCREMENT 10
typedef char SElemtype;
typedef int Status;
typedef struct {
    SElemtype *top;
    SElemtype *base;
    int stacksize;
} SqStack;

Status InitStack(SqStack *s) {
    s->base = (SElemtype *)malloc(STACK_INIT_SIZE * sizeof(SElemtype));
    if (!s->base)
        exit(-2);
    s->top = s->base;
    s->stacksize = STACK_INIT_SIZE;
    return 1;
}

Status Push(SqStack *s, SElemtype c) {
    if (s->top - s->base == s->stacksize) {
        s->base = (SElemtype *)realloc(s->base, (s->stacksize + STACKINCREMENT) *

```

```

sizeof(SElemtype));
    if (!s->base)
        exit(-2);
    s->top = s->base + s->stacksize;
    s->stacksize += STACKINCREMENT;
}
*s->top++ = c;
return 1;
}

```

```

Status Pop(SqStack *s) {
    if (s->top == s->base)
        return 0;
    --s->top;
    return 1;
}

```

```

Status PrintStack(SqStack *s) {
    int i = 0;
    while ((s->base + i) != s->top) {
        printf("%c", *(s->base + i));
        i++;
    }
    return 1;
}

```

```

Status ClearStack(SqStack *s) {
    s->top = s->base;
    return 1;
}

```

```

void LineEdit(SqStack *s, SElemtype c) {
    switch (c) {
        case '#':
            Pop(s);
            break;
        case '@':
            ClearStack(s);
            break;
        default:
            Push(s, c);
            break;
    }
}

```

```

// 表达式求值
#define STACK_INIT_SIZE 100
#define STACKINCREMENT 10
typedef int Status;
typedef char ElemType;
typedef struct {

```

```

    ElemType *base;
    ElemType *top;
    int stacksize;
} sqStack;

Status InitStack(sqStack &s) {
    s.base = (ElemType *)malloc(STACK_INIT_SIZE * sizeof(ElemType));
    if (!s.base)
        exit(1);
    s.top = s.base;
    s.stacksize = STACK_INIT_SIZE;
    return 0;
}

ElemType GetTop(sqStack s) {
    ElemType e;
    if (s.top == s.base)
        return 0;
    e = *(s.top - 1);
    return e;
}

Status Push(sqStack &s, ElemType e) {
    if (s.top - s.base >= s.stacksize) {
        s.base = (ElemType *)realloc(s.base, (s.stacksize + STACKINCREMENT) *
sizeof(ElemType));
        if (!s.base)
            exit(1);
        s.top = s.base + s.stacksize;
        s.stacksize += STACKINCREMENT;
    }
    *s.top++ = e;
    return 0;
}

Status Pop(sqStack &s, ElemType &e) {
    if (s.top == s.base)
        return 1;
    e = *--s.top;
    return 0;
}

int In(ElemType c) {
    if (c == '+' || c == '-' || c == '*' || c == '/' || c == '#' || c == '(' || c ==
')' || c == '[' || c == ']')
        return 1;
    else
        return 0;
}

char Precede(ElemType a, ElemType b) {

```

```

if (a == '+' || a == '-') {
    if (b == '+' || b == '-' || b == '>' || b == '#' || b == ')' || b == ']')
        return '>';
    else
        return '<';
}
if (a == '*' || a == '/') {
    if (b == '(' || b == '[')
        return '<';
    else
        return '>';
}
if (a == '(') {
    if (b == ')')
        return '=';
    else
        return '<';
}
if (a == '[') {
    if (b == ']')
        return '=';
    else
        return '<';
}
if (a == '#') {
    if (b == '#')
        return '=';
    else
        return '<';
}
}

```

```

ElemType Operate(ElemType a, ElemType x, ElemType b) {
    switch (x) {
        case '+':
            return a + b;
        case '-':
            return a - b;
        case '*':
            return a * b;
        case '/':
            return a / b;
    }
}

```

```

ElemType EvaluateExpression() {
    sqStack OPTR, OPND;
    char c, x;
    InitStack(OPTR);
    Push(OPTR, '#');
    InitStack(OPND);
}

```

```

    c = getchar();
    while (c != '#' || GetTop(OPTR) != '#') {
        if (!In(c)) {
            Push(OPND, c);
            c = getchar();
        } else
            switch (Precede(GetTop(OPTR), c)) {
                case '<':
                    Push(OPTR, c);
                    c = getchar();
                    break;
                case '=':
                    Pop(OPTR, x);
                    c = getchar();
                    break;
                case '>':
                    Pop(OPTR, x);
                    ElemType a, b;
                    Pop(OPND, b);
                    Pop(OPND, a);
                    Push(OPND, Operate(a, x, b));
                    break;
            }
    }
    return GetTop(OPND);
}

// 队列的顺序表示（循环队列）
#define MAXQSIZE 100
typedef int QElemType;
typedef struct {
    QElemType *base;
    int front;
    int rear;
} SqQueue;

Status InitQueue(SqQueue &Q) {
    Q.base = (QElemType *)malloc(MAXQSIZE * sizeof(QElemType));
    if (!Q.base)
        exit(-2);
    Q.front = Q.rear = 0;
    return 1;
}

Status EnQueue(SqQueue &Q, QElemType e) {
    if ((Q.rear + 1) % MAXQSIZE == Q.front)
        return 0;
    Q.base[Q.rear] = e;
    Q.rear = (Q.rear + 1) % MAXQSIZE;
    return 1;
}

```



```

Status DeQueue(SqQueue &Q, QElemType &e) {
    if (Q.front == Q.rear)
        return 0;
    e = Q.base[Q.front];
    Q.front = (Q.front + 1) % MAXQSIZE;
    return 1;
}

// 队列的链式表示和实现
typedef struct Qnode {
    ElemType data;
    struct Qnode *next;
} QNode;

typedef struct link_queue {
    QNode *front, *rear;
} Link_Queue;

LinkQueue *Init_LinkQueue(void) {
    LinkQueue *Q;
    QNode *p;
    p = (QNode *)malloc(sizeof(QNode));
    p->next = NULL;
    Q = (LinkQueue *)malloc(sizeof(LinkQueue));
    Q->front = Q->rear = p;
    return (Q);
}

Status Insert_CirQueue(LinkQueue *Q, ElemType e) {
    QNode *p = (QNode *)malloc(sizeof(QNode));
    if (!p)
        return 0;
    p->data = e;
    p->next = NULL;
    Q->rear->next = p;
    Q->rear = p;
    return 1;
}

Status Delete_LinkQueue(LinkQueue *Q, ElemType *x) {
    QNode *p;
    if (Q->front == Q->rear)
        return 0;
    p = Q->front->next;
    *x = p->data;
    Q->front->next = p->next;
    if (p == Q->rear)
        Q->rear = Q->front;
    free(p);
    return 1;
}

```

```

}

void Destroy_LinkQueue(LinkQueue *Q) {
    while (Q->front != NULL) {
        Q->rear = Q->front->next;
        free(Q->front);
        Q->front = Q->rear;
    }
}

int main() {
    // 数制转换测试
    printf("十进制159转换为八进制: ");
    conversion(159, 8);

    // 括号匹配测试
    printf("请输入括号序列进行匹配测试: ");
    if (Match_Brackets())
        printf("括号匹配成功\n");
    else
        printf("括号匹配失败\n");

    // 行编辑程序测试
    SqStack lineStack;
    InitStack(&lineStack);
    char c;
    printf("请输入字符（以#删除前一个字符，@清空当前行，EOF结束）:\n");
    while ((c = getchar()) != EOF) {
        LineEdit(&lineStack, c);
        if (c == '\n') {
            PrintStack(&lineStack);
            ClearStack(&lineStack);
            printf("\n");
        }
    }

    // 表达式求值测试
    printf("请输入算术表达式（以#结束）: ");
    char result = EvaluateExpression();
    printf("表达式求值结果: %c\n", result);

    // 循环队列测试
    SqQueue circularQueue;
    InitQueue(circularQueue);
    int value;
    if (EnQueue(circularQueue, 1))
        printf("元素1入队成功\n");
    if (DeQueue(circularQueue, value))
        printf("元素出队: %d\n", value);

    // 链队列测试

```

```

    LinkQueue *linkedQueue = Init_LinkQueue();
    if (Insert_CirQueue(linkedQueue, 2))
        printf("元素2入链队列成功\n");
    if (Delete_LinkQueue(linkedQueue, &value))
        printf("元素出链队列: %d\n", value);
    Destroy_LinkQueue(linkedQueue);

    return 0;
}

```

串

```

typedef struct {
    char ch[MAX_LENGTH];
    int length;
} String;

typedef struct {
    char *ch;
    int length;
} String;

// 定义链表节点
typedef struct Node {
    char data;
    struct Node *next;
} Node;

```

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// 串的定长顺序存储表示
#define MAX_STRLEN 256
typedef struct {
    char str[MAX_STRLEN];
    int length;
} StringType;

// 串的联结操作
int StrConcat(StringType *s, StringType t) {
    if ((s->length + t.length) > MAX_STRLEN) {
        return 0; // 联结后长度超出范围
    }
    for (int i = 0; i < t.length; i++) {
        s->str[s->length + i] = t.str[i]; // 串t联结到串s之后
    }
    s->length = s->length + t.length; // 修改联结后的串长度
    return 1;
}

// 求子串操作
int SubString(StringType s, int pos, int len, StringType *sub) {
    if (pos < 1 || pos > s.length || len < 0 || len > (s.length - pos + 1)) {
        return 0; // 参数非法
    }
    sub->length = len; // 求得子串长度
    for (int j = 0, k = pos - 1; j < len; k++, j++) {
        sub->str[j] = s.str[k]; // 逐个字符复制求得子串
    }
    return 1;
}

// Brute-Force模式匹配算法
int IndexString(StringType s, StringType t, int pos) {
    char *p, *q;
    int k, j;
    k = pos - 1;
    j = 0;
    p = s.str + pos - 1;
    q = t.str;

    while (k < s.length && j < t.length) {
        if (*p == *q) {
            p++;
            q++;
            k++;
            j++;
        }
    }
}

```

```

        } else {
            k = k - j + 1;
            j = 0;
            q = t.str;
            p = s.str + k;
        }
    }
    if (j == t.length) {
        return (k - t.length); // 匹配, 返回位置
    } else {
        return -1; // 不匹配, 返回-1
    }
}

```

// 比较两个采用顺序结构存储的串是否相等的算法

```

int StrCompare(StringType x, StringType y) {
    if (x.length != y.length) {
        return 0;
    }
    for (int i = 0; i < x.length; i++) {
        if (x.str[i] != y.str[i]) {
            return 0;
        }
    }
    return 1;
}

```

// 串的堆分配存储表示

```

typedef struct {
    char *ch;
    int length;
} HString;

```

// 串的联结操作（堆分配存储方式）

```

int StrConcatHeap(HString *T, HString *s1, HString *s2) {
    if (T->ch) {
        free(T->ch); // 释放旧空间
    }
    int t_len = s1->length + s2->length;
    T->ch = (char *)malloc((t_len + 1) * sizeof(char));
    if (!T->ch) {
        printf("系统空间不够, 申请空间失败 ! \n");
        return 0;
    }
    for (int j = 0; j < s1->length; j++) {
        T->ch[j] = s1->ch[j]; // 将串s1复制到串T中
    }
    for (int k = s1->length, j = 0; j < s2->length; k++, j++) {
        T->ch[k] = s2->ch[j]; // 将串s2复制到串T中
    }
    T->length = t_len;
}

```

```

    free(s1->ch);
    free(s2->ch);
    return 1;
}

// 串的链式存储表示
#define BLOCK_SIZE 4
typedef struct Blstrtype {
    char data[BLOCK_SIZE];
    struct Blstrtype *next;
} BNODE;

typedef struct {
    BNODE *head;
    int Strlen;
} Blstring;

// 模式匹配的改进算法（KMP算法）
#define Max_Strlen 1024
int next[Max_Strlen];

// 求next函数值的算法
void getNext(StringType t, int next[]) {
    int k = 0, j = 1;
    next[0] = -1;
    while (j < t.length) {
        if (k == -1 || t.str[k] == t.str[j]) {
            k++;
            j++;
            if (t.str[k] != t.str[j]) {
                next[j] = k;
            } else {
                next[j] = next[k];
            }
        } else {
            k = next[k];
        }
    }
}

// KMP算法进行模式匹配
int KMP_index(StringType s, StringType t) {
    int k = 0, j = 0;
    getNext(t, next);
    while (k < s.length && j < t.length) {
        if (j == -1 || s.str[k] == t.str[j]) {
            k++;
            j++;
        } else {
            j = next[j];
        }
    }
}

```

```

    }
    if (j >= t.length) {
        return (k - t.length); // 匹配, 返回位置
    } else {
        return -1; // 不匹配, 返回-1
    }
}

// 将T中第一次出现的与P相等的子串替换为S的算法
void StrReplace(char *T, char *P, char *S) {
    int T_len = strlen(T);
    int P_len = strlen(P);
    int S_len = strlen(S);
    char *pos = strstr(T, P);
    if (pos) {
        int shift = S_len - P_len;
        int move_len = T_len - (pos - T) - P_len;
        memmove(pos + S_len, pos + P_len, move_len + 1);
        memcpy(pos, S, S_len);
    }
}

int main() {
    // 定长顺序存储串的测试
    StringType s1 = {"hello", 5};
    StringType s2 = {"world", 5};
    StringType sub;
    if (StrConcat(&s1, s2)) {
        printf("联结后的串: %s\n", s1.str);
    }
    if (SubString(s1, 3, 3, &sub)) {
        printf("子串: %s\n", sub.str);
    }
    int index = IndexString(s1, s2, 1);
    printf("Brute-Force匹配结果: %d\n", index);

    // 堆分配存储串的测试
    HString h1 = {NULL, 0};
    HString h2 = {NULL, 0};
    HString hT = {NULL, 0};
    h1.ch = (char *)malloc(6 * sizeof(char));
    h2.ch = (char *)malloc(6 * sizeof(char));
    strcpy(h1.ch, "hello");
    strcpy(h2.ch, "world");
    h1.length = 5;
    h2.length = 5;
    if (StrConcatHeap(&hT, &h1, &h2)) {
        printf("堆分配联结后的串: %s\n", hT.ch);
    }

    // 链式存储串暂未进行详细测试, 可根据实际需求添加

```

```

// KMP算法测试
StringType kmp_s = {"abacabab", 8};
StringType kmp_t = {"abab", 4};
int kmp_index_result = KMP_index(kmp_s, kmp_t);
printf("KMP匹配结果: %d\n", kmp_index_result);

// 串替换测试
char replace_T[20] = "hello world";
char replace_P[6] = "world";
char replace_S[6] = "universe";
StrReplace(replace_T, replace_P, replace_S);
printf("替换后的串: %s\n", replace_T);

// 释放堆分配存储串的空间
free(h1.ch);
free(h2.ch);
free(hT.ch);

return 0;
}

```

数组和广义表

```

typedef struct {
    int row; // 非零元素所在的行
    int col; // 非零元素所在的列
    int value; // 非零元素的值
} Triple;

// 定义广义表节点的结构体
typedef struct GLNode {
    int tag; // 标志位, ATOM 表示原子, LIST 表示子表
    union {
        char atom; // 原子的值
        struct {
            struct GLNode *hp, *tp; // hp 指向表头, tp 指向表尾
        } ptr;
    };
} GLNode, *GList;

```



```

#include <stdio.h>
#include <stdlib.h>

// 定义三元组表结构
#define MAXSIZE 12500
typedef int ElemType;
typedef struct {
    int i, j;
    ElemType e;
} Triple;
typedef struct {
    Triple data[MAXSIZE + 1];
    int mu, nu, tu;
} TSMatrix;

// 按列序转置稀疏矩阵
int TransposeSMatrix(TSMatrix M, TSMatrix &T) {
    T.mu = M.nu;
    T.nu = M.mu;
    T.tu = M.tu;
    if (T.tu) {
        int q = 1;
        for (int col = 1; col <= M.nu; ++col) {
            for (int p = 1; p <= M.tu; ++p) {
                if (M.data[p].j == col) {
                    T.data[q].i = M.data[p].j;
                    T.data[q].j = M.data[p].i;
                    T.data[q].e = M.data[p].e;
                    ++q;
                }
            }
        }
    }
    return 1;
}

// 快速转置稀疏矩阵
int FastTransposeSMatrix(TSMatrix M, TSMatrix &T) {
    T.mu = M.nu;
    T.nu = M.mu;
    T.tu = M.tu;
    if (T.tu) {
        int num[MAXSIZE], cpot[MAXSIZE];
        for (int col = 1; col <= M.nu; ++col) num[col] = 0;
        for (int t = 1; t <= M.tu; ++t) ++num[M.data[t].j];
        cpot[1] = 1;
        for (int col = 2; col <= M.nu; ++col) cpot[col] = cpot[col - 1] + num[col - 1];
        for (int p = 1; p <= M.tu; ++p) {
            int col = M.data[p].j;

```

```

        int q = cpot[col];
        T.data[q].i = M.data[p].j;
        T.data[q].j = M.data[p].i;
        T.data[q].e = M.data[p].e;
        ++cpot[col];
    }
}
return 1;
}

// 定义广义表结点结构
typedef int elemtype;
typedef struct GLNode {
    int tag;
    union {
        elemtype value;
        struct {
            struct GLNode *hp, *tp;
        } ptr;
    } Gdata;
} GLNode;

// 创建广义表
GLNode* createGeneralizedList() {
    // 示例创建广义表 (a, (b, c, d))
    GLNode *list = (GLNode*)malloc(sizeof(GLNode));
    list->tag = 1;
    // 创建表头原子结点 a
    GLNode *headAtom = (GLNode*)malloc(sizeof(GLNode));
    headAtom->tag = 0;
    headAtom->Gdata.value = 'a';
    list->Gdata.ptr.hp = headAtom;
    // 创建表尾广义表 (b, c, d)
    GLNode *taillist = (GLNode*)malloc(sizeof(GLNode));
    taillist->tag = 1;
    // 创建 (b, c, d) 的表头原子结点 b
    GLNode *subHeadAtom = (GLNode*)malloc(sizeof(GLNode));
    subHeadAtom->tag = 0;
    subHeadAtom->Gdata.value = 'b';
    taillist->Gdata.ptr.hp = subHeadAtom;
    // 创建 (b, c, d) 的第二个原子结点 c
    GLNode *cAtom = (GLNode*)malloc(sizeof(GLNode));
    cAtom->tag = 0;
    cAtom->Gdata.value = 'c';
    subHeadAtom->Gdata.ptr.tp = cAtom;
    // 创建 (b, c, d) 的第三个原子结点 d
    GLNode *dAtom = (GLNode*)malloc(sizeof(GLNode));
    dAtom->tag = 0;
    dAtom->Gdata.value = 'd';
    cAtom->Gdata.ptr.tp = dAtom;
    dAtom->Gdata.ptr.tp = NULL;
}

```

```

    list->Gdata.ptr.tp = taillist;
    return list;
}

// 获取广义表表头
GLNode* getHead(GLNode *list) {
    if (list == NULL || list->tag == 0) {
        return NULL;
    }
    return list->Gdata.ptr.hp;
}

// 获取广义表表尾
GLNode* getTail(GLNode *list) {
    if (list == NULL || list->tag == 0) {
        return NULL;
    }
    return list->Gdata.ptr.tp;
}

// 释放广义表内存
void freeGeneralizedList(GLNode *list) {
    if (list == NULL) {
        return;
    }
    if (list->tag == 0) {
        free(list);
    } else {
        freeGeneralizedList(list->Gdata.ptr.hp);
        freeGeneralizedList(list->Gdata.ptr.tp);
        free(list);
    }
}

int main() {
    // 稀疏矩阵转置测试
    TSMatrix M = {
        .mu = 6,
        .nu = 7,
        .tu = 8,
        .data = {
            {0, 0, 0},
            {1, 2, 12},
            {1, 3, 9},
            {3, 1, -3},
            {3, 6, 14},
            {4, 3, 24},
            {5, 2, 18},
            {6, 1, 15},
            {6, 4, -7}
        }
    }
}

```

```

};
TSMatrix T;
if (TransposeSMatrix(M, T)) {
    printf("按列序转置后的矩阵: \n");
    for (int i = 1; i <= T.tu; ++i) {
        printf("%d %d %d\n", T.data[i].i, T.data[i].j, T.data[i].e);
    }
}
if (FastTransposeSMatrix(M, T)) {
    printf("快速转置后的矩阵: \n");
    for (int i = 1; i <= T.tu; ++i) {
        printf("%d %d %d\n", T.data[i].i, T.data[i].j, T.data[i].e);
    }
}

// 广义表操作测试
GLNode *generalizedList = createGeneralizedList();
GLNode *head = getHead(generalizedList);
GLNode *tail = getTail(generalizedList);
if (head) {
    if (head->tag == 0) {
        printf("表头原子值: %c\n", head->Gdata.value);
    } else {
        printf("表头为子表\n");
    }
}
if (tail) {
    printf("表尾为子表\n");
}
freeGeneralizedList(generalizedList);

return 0;
}

```

作业